



K

BIG DATA & DATA ENGINEERING SERIES

Apache Kafka for Beginners

A colorful, hands-on guide to event streaming —
architecture, partitions, consumer groups, replication, and
real Kafka CLI commands on Ubuntu.

Architecture & Concepts

Partitions & Parallelism

Consumer Groups

Kafka CLI Lab

Cheat Sheet

From zero to your first Kafka pipeline

Table of Contents

- [1 What is Apache Kafka?](#)
- [2 Kafka Core Architecture and Concepts](#)
- [3 Kafka Cluster and Brokers](#)
- [4 Topics and Partitions](#)
- [5 Why Partitions Enable Parallelism and Scalability](#)
- [6 Messages / Events](#)
- [7 Offsets](#)
- [8 Consumer Groups](#)
- [9 Replication](#)
- [10 Kafka Data Flow \(End-to-End\)](#)
- [11 Kafka CLI on Ubuntu](#)
- [12 Complete Hands-On Kafka CLI Lab](#)
- [13 Important Relationships \(Visual Summary\)](#)
- [14 Beginner Mistakes to Avoid](#)
- [15 Final Cheat Sheet](#)

SECTION 01

What is Apache Kafka?

Apache Kafka is an open-source **distributed event-streaming platform**. In plain terms, Kafka is a system that lets applications publish, store, and read streams of events (records) in real time, reliably, and at very large scale. Instead of one application calling another directly, applications write events to Kafka, and any number of other applications can read those events — independently, at their own pace.

The Problem Kafka Solves

Imagine an e-commerce company where the "Orders" service needs to notify the "Inventory", "Billing", "Shipping", and "Analytics" services every time an order is placed. Without Kafka, the Orders service would need direct connections to each of these systems — a tangled, fragile web of point-to-point calls. If one downstream system is slow or down, it can block or break the whole chain.

Kafka solves this by sitting in the middle as a **durable, high-throughput buffer**. The Orders service publishes an `order_created` event once, and every interested service reads that same event independently, whenever it is ready.

REAL-WORLD EXAMPLE

An e-commerce app produces an event every time an order is placed: `{ "order_id": 5001, "customer": "customer_101", "amount": 79.99 }`. The Inventory service consumes this event to reduce stock. The Billing service consumes it to charge the customer. The Analytics service consumes it to update dashboards. All three read the same event from Kafka, completely independently of each other and of the Orders service.

Kafka as an Event-Streaming Platform

Kafka is built around three core capabilities:

- **Publish and subscribe** to streams of events, similar to a message queue.
- **Store** streams of events durably, on disk, for as long as needed.
- **Process** streams of events as they occur, enabling real-time pipelines.

Common Real-World Use Cases

Use Case	How Kafka Helps
E-commerce order pipelines	Order events flow to inventory, billing, shipping, and analytics in real time.
Log and metrics aggregation	Every server/app streams logs into Kafka for centralized monitoring.
Fraud detection	Payment events are analyzed as they arrive to flag suspicious activity instantly.
Website activity tracking	Clicks, page views, and searches stream into Kafka for real-time personalization.
Data lake / warehouse ingestion	Kafka acts as the backbone feeding a Data Lake, HDFS, or a Data Warehouse.

Kafka Architecture at a Glance



Producers write events into the Kafka cluster; consumers read them independently — Kafka decouples the two sides completely.

Kafka vs. Traditional Message Queues

Aspect	Traditional Queue (e.g. RabbitMQ)	Apache Kafka
Message lifetime	Deleted once consumed	Retained for a configured period, re-readable
Consumers	Usually one consumer processes a message	Many independent consumer groups can each read everything
Ordering	Simple FIFO queue	Ordered per partition, scalable across partitions
Throughput	Moderate	Very high, built for large data streams
Use case fit	Task queues, RPC-style work	Event streaming, real-time pipelines, big data ingestion

REMEMBER

Kafka is not just a message queue — it is a distributed, persistent log of events that multiple independent applications can read from at their own pace.

SECTION 02

Kafka Core Architecture and Concepts

Before diving deeper, let's define every core Kafka term on its own, with a simple real-world example for each. These building blocks combine to form the full picture in the sections that follow.

● Producer ● Broker ● Topic ● Partition ● Consumer ● Consumer Group

Kafka Cluster

DEFINITION: A group of Kafka servers (brokers) working together as one system.

PURPOSE: Provides scalability and fault tolerance by spreading data across machines.

EXAMPLE: A retail company runs a 3-broker cluster so no single machine failure stops order processing.

Broker

DEFINITION: A single Kafka server that stores data and serves client requests.

PURPOSE: Handles reads/writes for the partitions it owns and replicates data for durability.

EXAMPLE: Broker 1 stores partition 0 of the "orders" topic and answers producer/consumer requests for it.

RELATION: A cluster is made of multiple brokers; each topic's partitions are spread across brokers.

Topic

DEFINITION: A named category/stream where events are published — like a table name for events.

PURPOSE: Organizes related events so producers and consumers agree on where to write/read.

EXAMPLE: An "orders" topic holds every order-created event for the whole company.

Partition

DEFINITION: An ordered, append-only log that a topic is split into.

PURPOSE: Enables parallelism — different partitions can be written and read at the same time.

EXAMPLE: The "orders" topic has 4 partitions, so 4 consumers can process orders simultaneously.

RELATION: A topic is a logical name; partitions are where the actual data physically lives.

Producer

DEFINITION: A client application that writes (publishes) events to a Kafka topic.

PURPOSE: Gets data into Kafka so downstream systems can consume it.

EXAMPLE: The checkout service produces an event every time a customer completes a purchase.

Consumer

DEFINITION: A client application that reads events from a Kafka topic.

PURPOSE: Gets data out of Kafka to act on it — store it, transform it, or trigger logic.

EXAMPLE: The billing service consumes order events and charges the customer's card.

Consumer Group

DEFINITION: A named set of consumers that cooperate to read a topic, splitting the partitions among themselves.

PURPOSE: Allows horizontal scaling of consumption without processing the same event twice within the group.

EXAMPLE: "billing-group" has 4 consumer instances sharing the 4 partitions of "orders".

Message / Event

DEFINITION: A single record written to a partition, made up of a key, a value, a timestamp, and optional headers.

PURPOSE: Represents "something that happened" — the unit of data Kafka moves around.

EXAMPLE: Key: customer_101, Value: "Order #5001 created"

Offset

DEFINITION: A sequential ID number identifying a message's position within a partition.

PURPOSE: Lets consumers track exactly which messages they have already read.

EXAMPLE: Offset 4 in partition 0 might be the 5th event ever written to that partition (offsets start at 0).

Replication

DEFINITION: The process of copying each partition onto multiple brokers.

PURPOSE: Protects against data loss and downtime if a broker fails.

EXAMPLE: With replication factor 3, partition 0 exists on three different brokers.

Leader

DEFINITION: The one replica of a partition that handles all reads and writes for it.

PURPOSE: Keeps writes consistent by funneling them through a single source of truth per partition.

EXAMPLE: Broker 1 is the leader for partition 0, so all producers/consumers of that partition talk to Broker 1.

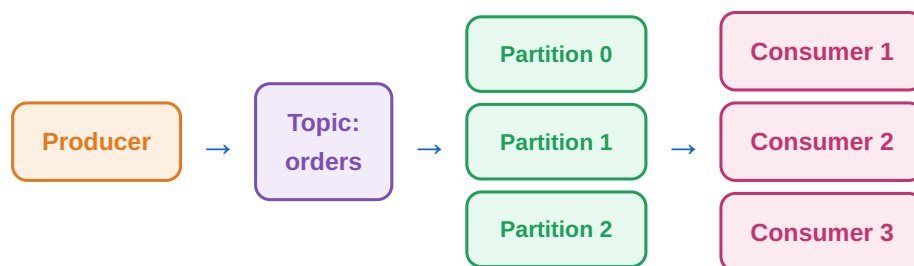
Follower

DEFINITION: A replica of a partition that copies data from the leader but does not directly serve client traffic.

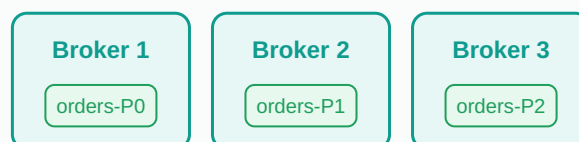
PURPOSE: Stands ready to become the new leader if the current leader broker fails.

EXAMPLE: Broker 2 and Broker 3 hold follower copies of partition 0, staying in sync with Broker 1.

Putting It Together: Producer → Topic → Partitions → Consumers



KAFKA CLUSTER (MULTIPLE BROKERS)



Each partition of the "orders" topic lives on a different broker, so the cluster shares the load.

SECTION 03

Kafka Cluster and Brokers

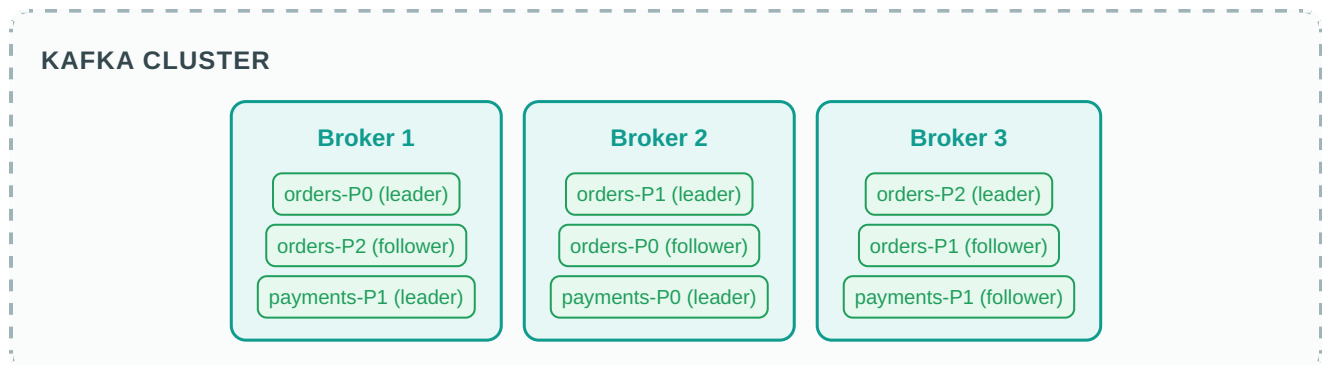
A **Kafka cluster** is simply a set of **brokers** — individual Kafka server processes — working together as one logical system. Clients (producers and consumers) don't need to know which exact broker holds which data; they connect to the cluster and Kafka routes requests to the right broker automatically.

Why Kafka Uses Multiple Brokers

- **Scalability** — more brokers means more machines to spread partitions and traffic across.
- **Fault tolerance** — if one broker goes down, replicas on other brokers keep the data available.
- **Throughput** — parallel reads/writes across brokers avoid a single machine becoming a bottleneck.

How Topics and Partitions Are Distributed

When a topic is created with multiple partitions, Kafka spreads those partitions across the available brokers in the cluster — it does not pile every partition onto one machine. Each partition also has replica copies placed on other brokers, according to the topic's replication factor.



Partitions and their replicas are spread across all three brokers — no single broker holds everything.

BEGINNER TIP

Think of a broker as a single "warehouse" and the cluster as the whole "logistics network" of warehouses. Data (partitions) is stored across multiple warehouses so the network keeps working even if one warehouse has a problem.

How Brokers Provide Scalability and Fault Tolerance

Benefit	How Multiple Brokers Deliver It
Scalability	More brokers = more partitions can be hosted = more parallel throughput.
Fault tolerance	Each partition's replicas live on different brokers, so one broker's failure doesn't lose data.
High availability	If a leader broker fails, a follower on another broker is promoted to leader automatically.
Load distribution	Client traffic spreads across many brokers instead of overwhelming one machine.

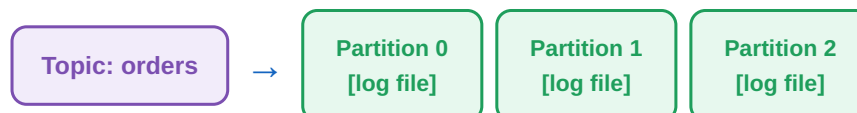
Topics and Partitions

What Is a Topic?

A **topic** is a named stream of events — the logical channel that producers write to and consumers read from, similar in spirit to a table name in a database. A topic by itself does not store data directly; it is really a container for one or more **partitions**, which are where the data physically lives.

What Is a Partition?

A **partition** is an ordered, append-only log of records. When a topic is created, you choose how many partitions it should have. Each partition is stored on disk on a broker, and new records are always appended to the end of the partition.



Why Kafka Uses Partitions

- They split a topic's data across multiple brokers, avoiding a single-machine bottleneck.
- They allow multiple consumers to read the same topic **in parallel**.
- They allow multiple producers to write to the same topic concurrently.

How Messages Are Stored Inside a Partition

Each partition is an immutable, append-only sequence. New records are always added to the end, and every record gets a sequential **offset** — its position number within that partition (covered in detail in Section 7).

0	1	2	3	4
Event A	Event B	Event C	Event D	Event E ← newest

Ordering Guarantees

IMPORTANT

Kafka guarantees **ordering only within a single partition**. Events written to partition 0 will always be read back in the same order they were written. Kafka does **not** guarantee any ordering *across* different partitions of the same topic — if partition 0 and partition 1 both receive events at the same time, there is no global order between them.

This is why the choice of a message's **key** matters: Kafka uses the key to decide which partition a message goes to. If you always use the same key (e.g. `customer_101`), every event for that customer lands in the same partition, and you get guaranteed per-customer ordering — but no ordering guarantee between different customers' events.

Scope	Ordering Guaranteed?
Within one partition	Yes — strict append order
Across partitions of the same topic	No global ordering
Across different topics	No relationship at all

SECTION 05

Why Partitions Enable Parallelism and Scalability

This is one of the most important ideas in Kafka. **A single partition can only be actively read by one consumer at a time within a given consumer group.** That means a topic with just one partition has a hard ceiling on how fast it can be consumed — no matter how many consumer instances you add, only one of them will ever be doing work.

IMPORTANT

One partition = one bottleneck. If "orders" had only 1 partition, adding more consumers would not speed up processing at all — the extra consumers would simply sit idle.

How Multiple Partitions Unlock Parallelism

By splitting a topic into several partitions, Kafka allows several consumers to each own one partition and process it independently and simultaneously. More partitions means more potential parallel workers.

```
# Topic: orders (4 partitions)
```

```
Partition 0 → Consumer 1  
Partition 1 → Consumer 2  
Partition 2 → Consumer 3  
Partition 3 → Consumer 4
```

All four consumers work at the same time, each handling its own slice of the traffic. This is how Kafka achieves high throughput — the work is divided, not duplicated.

Partitions Are Distributed Across Brokers

Partitions don't just enable parallel *consumption* — they also enable parallel *storage and I/O*, because each partition can live on a different broker. So scaling partitions also scales how much disk and network capacity a topic can use across the cluster.

Partitions, Consumers, Brokers, and Throughput

Factor	Effect on Throughput
More partitions	Potentially more parallel consumers and more distributed I/O across brokers
More brokers	More machines to spread partitions and network/disk load across
More consumers (within a group, up to partition count)	Higher parallel processing throughput
More consumers beyond partition count	No benefit — extra consumers sit idle (see Section 8)

REMEMBER

The number of partitions is the **upper limit** on how many consumers in a single consumer group can be actively working at once. Choosing the right partition count is one of the most important Kafka design decisions.

SECTION 06

Messages / Events

Everything that flows through Kafka is a **message**, also called a **record** or **event**. Each message written to a partition has a consistent structure:

Anatomy of a Kafka Message

KEY: Optional identifier used to decide which partition the message goes to, and often used for grouping (e.g. a customer ID).

VALUE: The actual payload/content of the event — the data itself.

TIMESTAMP: When the event was created or appended to the log.

HEADERS: Optional small key-value metadata pairs attached to the message (e.g. a trace ID), similar to HTTP headers.

SIMPLE EXAMPLE

Key: customer_101

Value: "Order #5001 created"

Timestamp: 2026-08-16T10:32:05Z

How Messages Are Appended to Partitions

When a producer sends a message, Kafka appends it to the end of the target partition's log. The message is not inserted, edited, or reordered — Kafka logs are strictly append-only. This simple design is a big part of why Kafka can handle such high write throughput.

Why Kafka Messages Are Persistent

Unlike a traditional queue that deletes a message once it's consumed, Kafka keeps messages on disk for a configurable retention period (for example, 7 days) or until a size limit is reached, regardless of whether they've been read. This means:

- Multiple different consumer groups can read the same events independently.
- A new consumer can be added later and still read historical events.
- If a consumer crashes, it can restart and re-read from where it left off (or from the beginning).

BEGINNER TIP

Kafka is often described as a "commit log" — think of it as a durable, ordered diary of everything that happened, that different readers can revisit at their own pace.

SECTION 07

Offsets

An **offset** is a sequential integer that identifies the position of a record within its partition. The first record ever written to a partition gets offset 0, the next gets offset 1, and so on. Offsets only ever increase — they are never reused, even if old messages are deleted by retention.

0	1	2	3	4
Event A	Event B	Event C	Event D	Event E

Partition 0 — the offset marks each record's position within *this specific partition*.

Offsets Are Specific to a Partition

IMPORTANT

An offset is **not** a global message ID across the whole topic. Offset 4 in Partition 0 and offset 4 in Partition 1 refer to two completely different, unrelated messages. Offsets only make sense together with a specific partition number.

How Consumers Use Offsets

As a consumer reads messages, it keeps track of the offset of the last message it successfully processed for each partition it owns. Periodically (or after each batch), the consumer **commits** that offset back to Kafka. This committed offset is the consumer's "bookmark".

What Happens When a Consumer Stops and Restarts

When a consumer starts up again after stopping, it looks up the last **committed offset** for its consumer group and partition, and resumes reading from right after that point — it does not need to re-read from the very beginning, and it does not lose track of where it was.

Offsets and Consumer Groups

Committed offsets are tracked **per consumer group, per partition** — not globally. This means two different consumer groups reading the same topic each maintain their own independent bookmark. Group A could be far ahead while Group B is still processing older messages, and neither affects the other.

Concept	Meaning
Offset	Position of a record within one partition
Current offset	The next offset a consumer will read
Committed offset	The last offset a consumer group has confirmed processing for a partition
Consumer lag	Difference between the latest offset in the partition and the committed offset

SECTION 08

Consumer Groups

A **consumer group** is a named set of consumer instances that work together to read a topic, dividing the topic's partitions among themselves. Every consumer belongs to a group, identified by a **group ID** (e.g. `billing-group`).

Why Consumer Groups Exist

Consumer groups let you scale out consumption horizontally: instead of one consumer struggling to keep up with a high-volume topic, several consumer instances in the same group split the partitions between them and process in parallel — while each message is still only handled once per group.

Partition Assignment Within a Group

IMPORTANT

Within the **same** consumer group, one partition can be assigned to only **one** consumer at a time. Two consumers in the same group will never process the same partition simultaneously.

TOPIC: ORDERS — 4 PARTITIONS

Consumer Group A

Consumer 1

Consumer 2

Consumer 3

Consumer 4



Partition 0

Partition 1

Partition 2

Partition 3

What Happens If Consumer 2 Stops?

If Consumer 2 leaves the group (crashes or shuts down), Kafka triggers a **rebalance**: Partition 1, which Consumer 2 owned, is reassigned to one of the remaining consumers in the group. No partition is left unattended — but that remaining consumer now has to handle two partitions' worth of work.

Consumer 1

now owns P0 + P1

Consumer 3

P2

Consumer 4

P3

More Consumers Than Partitions

If a group has more consumers than the topic has partitions, the extra consumers are assigned **no partitions at all** and sit idle, ready to take over if an active consumer fails.

Fewer Consumers Than Partitions

If a group has fewer consumers than partitions, some consumers are assigned more than one partition each, so all partitions are still covered — just with less parallelism.

Scenario	Result
Consumers = Partitions	Perfect 1-to-1 mapping; maximum parallelism
Consumers < Partitions	Some consumers handle multiple partitions
Consumers > Partitions	Extra consumers are idle, acting as standby

Offsets Per Consumer Group

As covered in Section 7, each consumer group tracks its own committed offsets per partition. This is what allows completely different applications — say, "billing-group" and "analytics-group" — to consume the exact same topic independently, each at its own pace, without interfering with one another.

Consumer Rebalancing (Beginner Level)

A **rebalance** is the process where Kafka redistributes partitions among the active consumers of a group. It's triggered whenever a consumer joins or leaves the group (for example, on startup, shutdown, or crash, or when the topic's partition count changes). During a rebalance there is a brief pause in processing while partitions are reassigned.

BEGINNER TIP

Frequent, unexpected rebalances are usually a sign something is wrong (e.g. consumers crashing or taking too long to process a batch). A healthy Kafka application should rebalance rarely.

SECTION 09

Replication

Replication means Kafka keeps multiple copies of each partition on different brokers. The number of copies is controlled by the topic's **replication factor** — for example, a replication factor of 3 means each partition has three copies spread across three different brokers.

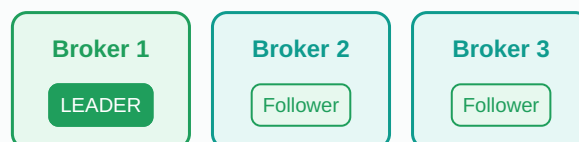
Why Kafka Replicates Partitions

- **Fault tolerance** — if the broker holding one copy fails, other copies still have the data.
- **High availability** — the topic keeps serving reads and writes even during a broker outage.
- **Durability** — data isn't lost just because a single disk or machine fails.

Leader and Follower Replicas

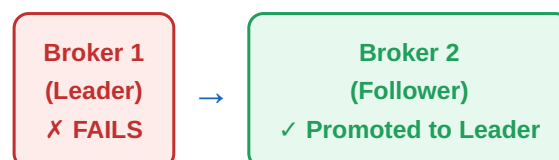
For each partition, exactly one replica is elected the **leader**; all producer writes and consumer reads for that partition go through the leader. The other replicas are **followers**, which continuously copy data from the leader to stay in sync, but do not directly serve client traffic under normal conditions.

PARTITION 0 — REPLICATION FACTOR 3



What Happens When a Broker Fails

If the broker hosting the leader replica goes down, Kafka automatically promotes one of the in-sync follower replicas (on a different broker) to become the new leader. Producers and consumers are redirected to the new leader, usually with only a brief interruption — no data is lost as long as at least one in-sync replica survived the failure.



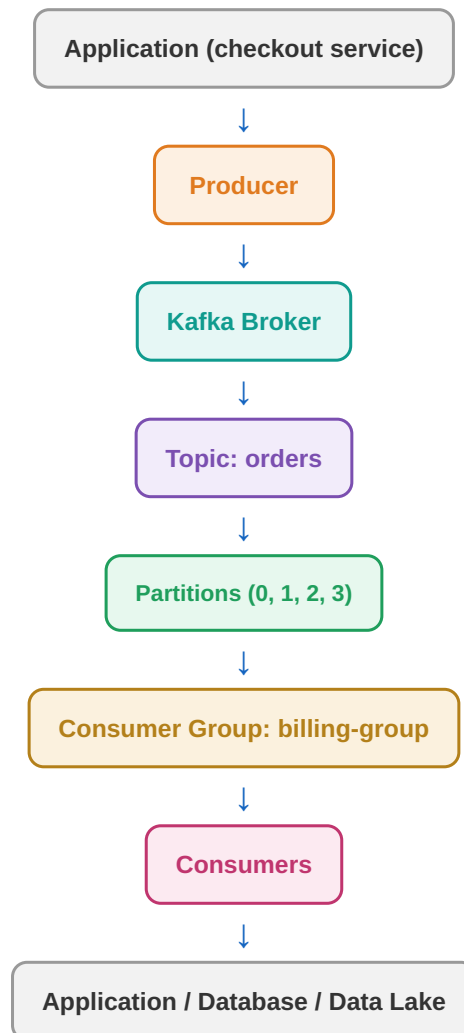
REMEMBER

Replication is about **durability and availability** (surviving failures). Partitioning is about **parallelism and scalability** (spreading the workload). They solve different problems and both are needed for a production-grade Kafka setup.

SECTION 10

Kafka Data Flow (End-to-End)

Let's trace a single event's complete journey through Kafka, from creation to final use.



Step-by-Step Walkthrough

1. **An event happens** — a customer completes checkout in the application.
2. **The producer** packages the event (key, value, timestamp) and sends it to Kafka.
3. **A broker** receives the write request and, based on the key, routes it to the correct partition.
4. **The topic's partition** appends the event as a new record with the next available offset.
5. **Replication** copies the new record to follower brokers for durability (Section 9).
6. **A consumer group** such as "billing-group" has consumers assigned to partitions of this topic.
7. **Each consumer** polls its assigned partition(s), reads new records past its committed offset, and processes them.
8. **The consumer commits** its new offset once processing succeeds, bookmarking its progress.
9. **The result** is written onward — to a database, a data lake, a dashboard, or triggers another action (e.g. charging the customer).

REAL-WORLD EXAMPLE

A customer buys a laptop. The Orders service (producer) sends an `order_created` event to the "orders" topic. Kafka appends it to partition 2 with offset 918. The Billing consumer group picks it up, charges the card, and writes a confirmation back to the database — while, completely separately, the Analytics consumer group reads the very same event to update a sales dashboard.

SECTION 11

Kafka CLI on Ubuntu

This section assumes Kafka has been downloaded and extracted on an Ubuntu machine (for example, into `~/kafka`), and that its `bin/` directory is on your PATH or you are running commands from inside it.

Kafka Server / Environment

Start ZooKeeper or KRaft controller (environment prep)

Older Kafka versions require ZooKeeper to be running before the broker starts; newer versions can run in KRaft mode without ZooKeeper. Check which mode your Kafka distribution uses.

```
# ZooKeeper mode (older Kafka)
bin/zookeeper-server-start.sh config/zookeeper.properties
```

Start the Kafka broker

```
# Start Kafka server using the default server config
bin/kafka-server-start.sh config/server.properties
```

What it does: Launches a Kafka broker process using the settings in `server.properties` (broker ID, listener port, log directory, etc.). Leave this terminal running — it is your live broker.

Checking Kafka processes are running

```
# Check for running Kafka / Java processes
jps

1234 Kafka
1180 QuorumPeerMain
1502 Jps
```

What it does: `jps` lists running Java processes. "Kafka" confirms the broker is up; "QuorumPeerMain" confirms ZooKeeper is up (if used).

Topics

Create a topic

```
kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --partitions 4 \
  --replication-factor 1
```

Created topic orders.

Key options:

- `--bootstrap-server` — address of a broker to connect to (host:port).
- `--topic` — the name of the topic to create.
- `--partitions` — how many partitions to split the topic into.
- `--replication-factor` — how many copies of each partition to keep (must be $\leq$ number of brokers).

List topics

```
kafka-topics.sh --list --bootstrap-server localhost:9092
```

orders
payments

Describe a topic

```
kafka-topics.sh --describe \
  --topic orders \
  --bootstrap-server localhost:9092
```

```
Topic: orders PartitionCount: 4 ReplicationFactor: 1
Partition: 0 Leader: 0 Replicas: 0 Isr: 0
Partition: 1 Leader: 0 Replicas: 0 Isr: 0
Partition: 2 Leader: 0 Replicas: 0 Isr: 0
Partition: 3 Leader: 0 Replicas: 0 Isr: 0
```

What it does: Shows the partition count, replication factor, and for each partition, which broker is the leader and which brokers hold in-sync replicas (Isr).

Delete a topic

```
kafka-topics.sh --delete \  
  --topic orders \  
  --bootstrap-server localhost:9092
```

BEGINNER TIP

Deletion may be asynchronous — the topic can briefly still appear in `--list` output while Kafka finishes cleaning it up in the background.

Producer

`kafka-console-producer.sh` lets you type messages directly into the terminal and send them to a topic — great for testing.

```
kafka-console-producer.sh \  
  --bootstrap-server localhost:9092 \  
  --topic orders  
  
> Order #5001 created  
> Order #5002 created  
> Order #5003 created
```

What happens: Each line you type and press Enter on is sent as one message to the "orders" topic, appended to a partition, and gets its own offset. Press `Ctrl+C` to exit.

Consumer

`kafka-console-consumer.sh` reads and prints messages from a topic to the terminal.

Consume new messages only

```
kafka-console-consumer.sh \  
  --bootstrap-server localhost:9092 \  
  --topic orders
```

By default, the consumer only reads messages produced *after* it started.

Read from the beginning

```
● ● ●

kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning

Order #5001 created
Order #5002 created
Order #5003 created
```

`--from-beginning` tells the consumer to start reading from offset 0 of every partition, instead of only new messages.

Specifying a consumer group

```
● ● ●

kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --group billing-group
```

Adding `--group` makes this consumer part of a named consumer group, so its offsets are tracked and it shares partitions with any other consumers using the same group name.

Consumer Groups

List consumer groups

```
● ● ●

kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --list

billing-group
analytics-group
```

Describe a consumer group (offsets and lag)



```
kafka-consumer-groups.sh \  
  --bootstrap-server localhost:9092 \  
  --describe \  
  --group billing-group
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
billing-group	orders	0	120	120	0
billing-group	orders	1	98	101	3
billing-group	orders	2	75	75	0
billing-group	orders	3	60	60	0

What it does: Shows, per partition, how far the group has read (CURRENT-OFFSET), how many messages exist in total (LOG-END-OFFSET), and how far behind it is (LAG) — plus which consumer instance owns each partition.

REMEMBER — CONSUMER LAG

Consumer lag is the gap between the latest offset written to a partition and the offset a consumer group has committed. High or growing lag means the consumer group is falling behind the incoming data rate — a key metric to monitor in production.

SECTION 12

Complete Hands-On Kafka CLI Lab

This mini lab walks through a full experiment on Ubuntu: create a topic, produce and consume messages, form a consumer group with multiple consumers, and observe partition assignment, offsets, and lag. Run each step in its own terminal window, in order.

1 Start Kafka

In terminal 1 (and ZooKeeper first if required):

```
bin/kafka-server-start.sh config/server.properties
```

2 Create the "orders" topic

```
kafka-topics.sh --create --bootstrap-server localhost:9092 --topic orders --  
partitions 4 --replication-factor 1
```

3 Create several partitions

We already created 4 partitions above via `--partitions 4`. You can increase (never decrease) partitions later with `--alter`:

```
kafka-topics.sh --alter --topic orders --partitions 6 --bootstrap-server localhost:  
9092
```

4 Describe the topic

```
kafka-topics.sh --describe --topic orders --bootstrap-server localhost:9092
```

Confirm you see 4 partitions listed, each with a leader broker.

5 Start a console producer (terminal 2)

```
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic orders
```

6 Send several order events

```
> Order #5001 created  
> Order #5002 created  
> Order #5003 created  
> Order #5004 created
```

7 Start a console consumer (terminal 3)

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders --from-beginning
```

8 Read messages

You should see all previously produced order events print in this terminal, read from the beginning of each partition.

9 Create a consumer group

Stop the consumer above (`Ctrl+C`) and restart it with a named group so it's trackable and shareable:

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders --group order-processors
```

10 Start multiple consumers in the same group

Open terminals 4 and 5, and run the *same* command with the *same* `--group order-processors` in each. Now three consumer instances (terminals 3, 4, 5) are all part of "order-processors".

11 Observe partition assignment

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group order-processors
```

The output's CONSUMER-ID / HOST columns show which of your three terminal instances owns which partition — with 4 partitions and 3 consumers, one consumer will own 2 partitions.

12 Check consumer group offsets

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group order-processors
```

Look at the CURRENT-OFFSET column for each partition — this is the group's bookmark.

13 Check consumer lag

In the same `--describe` output, the LAG column shows how far behind each partition's consumer is from the latest produced message.

14 Stop one consumer and observe the rebalance

Press `Ctrl+C` in one of the three consumer terminals, then re-run `--describe`. You'll see that consumer's partition(s) have been reassigned to the remaining two consumers — a live rebalance.

15 Produce more messages

Go back to the producer terminal and send a few more order events.

```
> Order #5005 created  
> Order #5006 created
```

16 Observe parallel processing

Watch the two remaining consumer terminals — new messages should appear split between them, based on which partitions each one now owns, confirming that partitions are what makes parallel consumption possible.

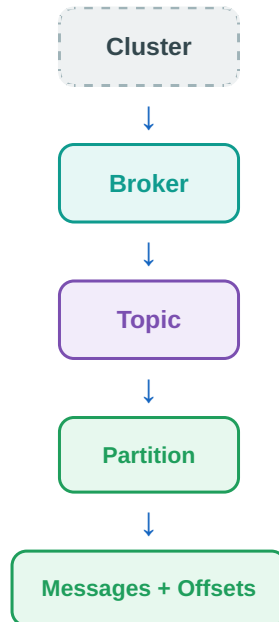
BEGINNER TIP

Run this whole lab again with `--partitions 1` instead of 4, and try adding multiple consumers to the same group. You'll see that only one consumer ever does any work — a hands-on demonstration of Section 5.

SECTION 13

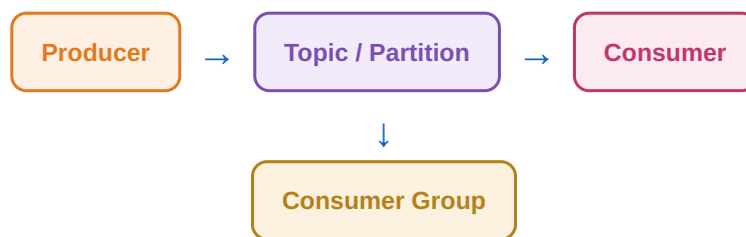
Important Relationships (Visual Summary)

The Structural Hierarchy



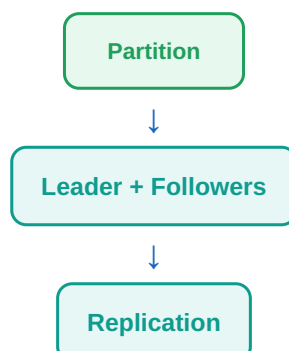
A cluster contains brokers; brokers host topic partitions; partitions contain ordered, offset-numbered messages.

The Data-Flow Relationship



Producers write to topic partitions; consumers read from them as members of a consumer group.

The Reliability Relationship



Each partition has one leader and multiple follower replicas, kept in sync through replication.

REMEMBER — THE THREE RELATIONSHIPS

Structure (cluster → broker → topic → partition) defines where data lives. **Flow** (producer → partition → consumer group → consumer) defines how data moves. **Reliability** (leader + followers → replication) defines how data survives failure.

SECTION 14

Beginner Mistakes to Avoid

- X Topic ≠ Partition.** A topic is a logical name; a partition is one physical, ordered log that a topic is split into. A topic can have many partitions.

- X Broker ≠ Cluster.** A broker is one server. A cluster is the whole group of brokers working together.

- X Consumer ≠ Consumer Group.** A consumer is a single client instance. A consumer group is a named team of consumers sharing the work of reading a topic.

- X An offset is not a global Kafka message ID.** It only identifies a message's position within one specific partition — the same offset number means something different in each partition.

- X More consumers do not always mean more parallelism.** Parallelism is capped by the number of partitions. Extra consumers beyond that sit idle.

- X A single partition cannot be actively processed by multiple consumers in the same consumer group simultaneously.** Only one consumer per group owns a given partition at any moment.

- X Replication is different from partitioning.** Partitioning splits data for parallelism; replication copies data for fault tolerance. They solve different problems.

- X Producers send records; consumers read records.** They are separate, decoupled roles — a producer never "pushes" data directly to a consumer.

- X Kafka does not automatically guarantee every message is processed exactly once.** Achieving exactly-once processing requires deliberate configuration (idempotent producers, transactions) — the default behavior can allow duplicates on retries.

Final Cheat Sheet

Core Concepts

- **Cluster** — group of brokers
- **Broker** — one Kafka server
- **Topic** — named event stream
- **Partition** — ordered log a topic is split into
- **Producer** — writes events
- **Consumer** — reads events
- **Consumer Group** — consumers sharing partitions
- **Message/Event** — key + value + timestamp + headers

Partition Rules

- Ordering guaranteed only within one partition
- No global ordering across partitions
- More partitions = more possible parallelism
- Partitions spread across brokers
- Same key → same partition, every time

Offset Rules

- Offset = position within one partition
- Offsets increase, never reused
- Not a global message ID
- Consumers commit offsets as bookmarks
- Restarting resumes from last committed offset

Consumer Group Rules

- 1 partition → only 1 consumer per group at a time
- Consumers > partitions → some sit idle
- Consumers < partitions → some own multiple partitions
- Offsets tracked per group, per partition
- Rebalance happens on join/leave

Replication Concepts

- Replication factor = number of copies
- Leader handles all reads/writes for a partition
- Followers copy from the leader, stay in sync
- Leader failure → a follower is promoted
- Protects against broker/disk failure

Most Important CLI Commands

- `kafka-topics.sh --create/--list/--describe/--delete`
- `kafka-console-producer.sh`
- `kafka-console-consumer.sh --from-beginning`
- `kafka-consumer-groups.sh --list/--describe`
- `jps` — check running processes

Important Command Options

- `--bootstrap-server` — broker address
- `--topic` — topic name
- `--partitions` — number of partitions
- `--replication-factor` — number of replicas
- `--group` — consumer group name
- `--from-beginning` — read from offset 0

Troubleshooting Commands

- `kafka-topics.sh --describe --topic X` — check partitions/leaders
- `kafka-consumer-groups.sh --describe --group X` — check lag
- `jps` — confirm broker/ZooKeeper are alive
- Check `logs/server.log` for broker errors
- High LAG → consumers are falling behind; consider more consumers (up to partition count)

WHERE TO GO NEXT

With these fundamentals solid, the natural next steps are: writing real Producer/Consumer code (Java, Python with `confluent-kafka` or `kafka-python`), exploring Kafka Connect for ingesting data from databases, and Kafka Streams / `ksqlDB` for real-time processing on top of your topics.